ASSIGNMENT 1

INSY6211

Student Number: ST10493664

Name: Reabetswe Tebogo Fafudi

Module Code: INSY6211

Class Group: Group 4

Table of Contents

Project Access 2
Question 1 – ACRMS Report 3
Question 2 – Potential Stakeholders 6
Question 3 – Decomposition Technique and Use Case Description 7
Question 4 – Domain Model Class Diagram 10
References 11

Project Access

GitHub Repo Link: https://github.com/reabetswef/ST10493664-INSY6211-Assignment-1.git

Question 1 – ACRMS Report

A Case for Structured Development: The Role of Systems Analysis and Design in the Airport Customer Relations Management System

1. What is Systems Analysis and Design?

Systems Analysis and Design (SAD) is simply a structured way of solving business problems with technology (Satzinger, Jackson and Burd, 2016). In the Analysis part, we focus on understanding the problem. We figure out exactly what the airport needs by talking to the people who will use the system and writing down their requirements (Kendall and Kendall, 2020). In the Design part, we focus on the solution. We decide exactly how the new system will work, including the software, data, and screens (Dennis, Wixom and Tegarden, 2015). For the new Airport Customer Relations Management System (ACRMS), SAD gives us the plan we need to turn your ideas about flight info, airport navigation, and emergency alerts into a reliable and easy-to-use system.

2. Why Getting SAD Right Matters for the ACRMS

Doing this properly is very important for the ACRMS because an airport is a busy, high-pressure place.

- **It keeps everyone happy**: Different people will use this system, like tourists, airport staff, and medics. SAD gives us tools to balance their different needs, so the system works well for everyone. This approach is critical for stakeholder alignment, which is a known success factor in complex information system projects (Bourgeois, 2019).
- **It reduces risk**: Proper planning helps us find problems early. It's much cheaper to fix a misunderstanding on paper now than to fix a system crash after it's been built. This reflects the principle that the cost of fixing errors increases exponentially the later they are found in the development lifecycle (Boehm, 1981).
- **It covers the important details**: It makes sure we don't forget the behind-the-scenes stuff, like making sure the system is reliable during a medical emergency, the GPS is accurate, and the kiosks can handle hundreds of people at once. This focus on non-functional requirements, such as performance and reliability, is essential for mission-critical systems like those in airport operations (Stair and Reynolds, 2020).

3. What We'll Do During the Analysis Phase

This phase is all about research and planning. Here's what we'll do to build a strong foundation:

- **Talk to the right people**: We'll identify the key users (passengers, medical teams, airport management) and use interviews and surveys to understand exactly what they need from the flight info, navigation, and emergency features. Requirement gathering through stakeholder analysis is a foundational step in system design (Wiegers and Beatty, 2013).
- **Map out the scenarios**: We'll use a technique called "event decomposition" to list every single thing the system needs to do. For example, what should happen when a

"passenger asks for medical help" or when a "flight is delayed"? We'll map out the system's response for each event. Event decomposition is a robust technique for identifying use cases by focusing on external events that a system must respond to (Satzinger, Jackson and Burd, 2016).

- **Create a data map**: We'll build a simple diagram showing the key pieces of information the system needs to manage, like Passenger, Flight, and MedicalEmergency, and how they are all connected. This is typically represented in an Entity-Relationship Diagram (ERD), which forms the conceptual basis for database design (Hoffer, Venkataraman and Topi, 2019).
- **Write clear instructions**: For every action the system must perform, we'll write a clear description. This covers the basic steps, what to do if something goes wrong, and what needs to happen before and after. These descriptions, known as use cases, are essential for capturing functional requirements and defining system behaviour (Cockburn, 2001).

4. What We'll Do During the Design Phase

Now we take the research from the analysis phase and start building the blueprint for the actual software.

- **Break it down into parts**: To keep things organized, we'll split the system into smaller, logical modules. This will likely include a Navigation part, an Emergency Services connector, a Flight Info updater, and a User Management section. This modular design, a form of systems architecture, improves maintainability and allows for parallel development (Dennis, Wixom and Tegarden, 2015).
- **Design the screens**: We'll create simple mock-ups of what the airport kiosks and the mobile app will look like. We'll focus on making it easy to use, ensuring that critical features like the "call for medical help" button is big, obvious, and easy to find. This user interface (UI) design process, often involving prototyping, is critical for usability and user adoption (Sharp, Rogers and Preece, 2019).
- **Design the database**: We'll design the behind-the-scenes structure for storing all the information. This means figuring out the best way to save and quickly find things like flight updates, locations of shops and gates, and details of an active emergency. A well-structured database schema, typically in a relational model, is crucial for data integrity and retrieval efficiency (Connolly and Begg, 2015).
- **Plan how parts talk to each other**: We'll map out how the different parts of the system (like the app and the navigation engine) will communicate. We'll also plan how the whole system will connect to the airport's existing flight data and emergency dispatch networks. This involves defining application programming interfaces (APIs) and integration architectures, which are key to ensuring system interoperability (Bass, Clements and Kazman, 2013).

5. Why We Recommend an Iterative Approach

Instead of building everything in one long sequence, we suggest an iterative approach,

which means building the system in repeated cycles. This is the best way to go for the ACRMS for a few reasons:

- **Test the tricky parts first**: The most complex feature is linking to medical services. By building a simple working version of this early on, we can test it thoroughly to make sure it's reliable before we build the rest of the system. This is a core principle of iterative development, where high-risk elements are addressed early in the project (Larman, 2004).
- **Adapt to changes easily**: Feedback might show that tourists find the GPS directions confusing. With an iterative approach, we can improve the design in the next cycle without throwing the whole project off track. Iterative methods like Agile are designed to accommodate changing requirements, a common challenge in software projects (Pressman, 2014).
- **Get feedback as we go**: This method lets users see and try parts of the system early. Their feedback helps us make sure the final product truly meets their needs, which leads to a better result and fewer expensive fixes after launch. User involvement through iterative prototyping significantly increases the likelihood of system success (Boehm and Turner, 2004).

6. Conclusion

In short, building the ACRMS is about more than just writing code. It’s a journey from understanding your needs (Analysis) to planning the solution (Design). The steps we've outlined such as defining requirements, designing the architecture, and prototyping the screens aren't just paperwork. They are the essential steps that make sure the final system is reliable, can grow with your needs, and is simple for passengers to use (Avison and Fitzgerald, 2006). Our commitment to building in cycles means we can handle the challenge of combining flight data, GPS, and emergency alerts, all while staying flexible to your feedback. We have the skills to put this plan into action, and we are confident we can deliver a system that will improve the passenger experience and be a valuable asset for the airport.

Question 2 – Potential Stakeholders

Q.2.1)

1. Airport Management / Airport Authority

- **Contribution to Requirements Gathering:** They provide the strategic vision and high-level business objectives for the system. They will clarify the airport's goals for customer satisfaction, define budget constraints, and outline the expected return on investment. Their input ensures the system aligns with the airport's overall direction.

2. Airline Representatives

- **Contribution to Requirements Gathering:** Airlines are key partners who will share data regarding flight schedules, delays, and passenger connections. They can specify requirements for how flight information should be displayed and how the system should handle disruptions, ensuring smooth communication between the airport and its airline tenants.

3. Passengers (End-Users)

- **Contribution to Requirements Gathering:** As the primary users of the system, passengers provide essential feedback on usability, desired features, and pain points. Through surveys, interviews, and usability testing, they help define requirements for intuitive navigation, clear flight information displays, and easy access to emergency services.

4. Ground Staff and Customer Service Agents

- **Contribution to Requirements Gathering:** These frontline employees interact with passengers daily and understand operational realities. They can identify practical needs such as how quickly emergency alerts must be received, what information is most helpful for assisting lost passengers, and how the system should integrate with their existing workflows.

5. Medical and Emergency Services Personnel

- **Contribution to Requirements Gathering:** As the system must provide "instant connection to medical and ambulatory services," these stakeholders are critical. They will specify requirements for alert accuracy, location precision, response time expectations, and what information they need immediately when responding to an emergency (e.g., passenger location, nature of the emergency)

Question 3 – Decomposition Technique and Use Case Description

Q.3.1)

Event 1

- Event Type: External Event
- Checklist Consideration: User requests information or assistance
- Event Description: Passenger initiates a request for medical or ambulatory assistance through the kiosk or mobile application interface.
- Use Case Name: Connect to Emergency Services

Event 2

- Event Type: External Event
- Checklist Consideration: External agent (airline system) sends data or updates to the system
- Event Description: An airline updates flight information such as delays, cancellations, or gate assignments, which must be reflected in the ACRMS.
- Use Case Name: Update Flight Information

Event 3

- Event Type: External Event
- Checklist Consideration: User requests information or assistance
- Event Description: Passenger requests turn-by-turn or map-based navigation to find an amenity (e.g., restaurant, restroom, shop) or a specific boarding gate.
- Use Case Name: Provide GPS Navigation Guidance

Event 4

- Event Type: External Event
- Checklist Consideration: User submits data to the system
- Event Description: Passenger submits feedback, a complaint, or a compliment regarding airport services, amenities, or overall experience.
- Use Case Name: Log Customer Feedback

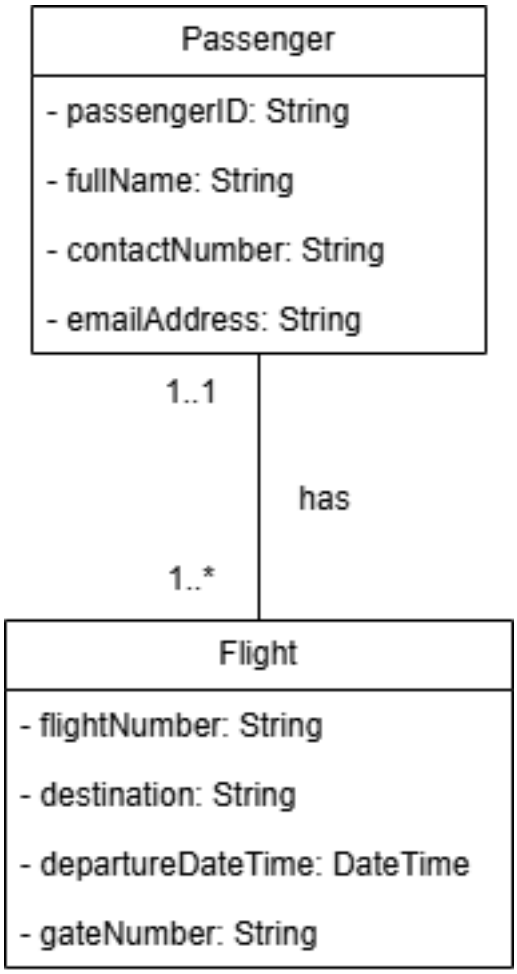
Q.3.2)

Element	Description
Use Case Name	Connect to Emergency Services
Use Case ID	UC-01
Priority	High (Critical)
Actors	Passenger (Primary), Medical Dispatcher (Secondary), Ambulatory Services (Secondary)
Stakeholders	Passenger (needs immediate, accurate assistance); Airport Management (requires reliable emergency response logging); Medical Personnel (need precise location and situation details)
Brief Description	This use case describes how a passenger can instantly request medical or ambulatory assistance through the ACRMS kiosk or mobile application, connecting them to appropriate emergency services.
Preconditions	1. The ACRMS is operational. 2. The kiosk or mobile device has connectivity to the airport's network. 3. Emergency services dispatch is available and integrated with the system.
Trigger	Passenger selects "Medical Emergency" or "Ambulatory Services" option on the kiosk or mobile app interface.
Main Success Scenario	1. Passenger selects the emergency assistance option. 2. System prompts passenger to confirm the nature of the emergency (e.g., cardiac, injury, general distress). 3. System automatically captures the passenger's precise location using GPS (mobile) or kiosk ID. 4. System establishes connection to the nearest available medical dispatcher. 5. System transmits location data and emergency details to the dispatcher. 6. System confirms to passenger that help is on the way and displays estimated arrival time. 7. System logs the emergency request with timestamp and response details.
Alternative Flows	**A1: Connection failure** 1. If system cannot connect to dispatcher immediately, it displays emergency contact numbers and alerts airport staff to respond manually. **A2: Passenger cancels request** 1. Passenger may cancel within 10 seconds of initiating request.

	2. System confirms cancellation with dispatcher if already notified. **A3: False alarm** 1. Dispatcher can mark request as resolved if no assistance needed.
Postconditions	1. Emergency services have been notified and dispatched to the correct location. 2. The incident is recorded in the system for reporting and analysis. 3. Passenger has received confirmation that assistance is en route.
Non-Functional Requirements	1. The connection to emergency services must be established within 3 seconds. 2. System must remain operational 24/7 with 99.9% uptime. 3. Location accuracy must be within 5 meters. 4. The emergency interface must be accessible with minimal navigation (maximum 2 taps/clicks).

Question 4 – Domain Model Class Diagram

Q.4.1)



Notes on Domain Model Design

This domain model follows the conceptual modelling approach where classes represent real-world entities in the problem domain. The relationship captures that a passenger may have multiple flights booked simultaneously, which is essential for the ACRMS to provide relevant flight information and navigation assistance for all of a passenger's upcoming departures.

The attributes selected are plausible for an airport customer relations context, supporting features such as:

- Personalized flight updates
- Emergency contact capabilities
- Efficient passenger identification at kiosks

References

Avison, D. and Fitzgerald, G., 2006. *Information Systems Development: Methodologies, Techniques and Tools*. 4th ed. London: McGraw-Hill.

Bass, L., Clements, P. and Kazman, R., 2013. *Software Architecture in Practice*. 3rd ed. Upper Saddle River, NJ: Addison-Wesley.

Boehm, B. and Turner, R., 2004. *Balancing Agility and Discipline: A Guide for the Perplexed*. Boston, MA: Addison-Wesley.

Boehm, B.W., 1981. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall.

Bourgeois, D.T., 2019. *Information Systems for Business and Beyond*. Arlington, VA: Saylor Academy.

Cockburn, A., 2001. *Writing Effective Use Cases*. Boston, MA: Addison-Wesley.

Connolly, T. and Begg, C., 2015. *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Boston, MA: Pearson.

Dennis, A., Wixom, B.H. and Tegarden, D., 2015. *Systems Analysis and Design: An Object-Oriented Approach with UML*. 5th ed. Hoboken, NJ: John Wiley & Sons.

GeeksforGeeks (2024) *Unified Modeling Language (UML) Class Diagrams*, Available at: https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/ (Accessed: 22 March 2026).

Hoffer, J.A., Venkataraman, R. and Topi, H., 2019. *Modern Database Management*. 13th ed. New York: Pearson.

Kendall, K.E. and Kendall, J.E., 2020. *Systems Analysis and Design*. 10th ed. New York: Pearson.

Larman, C., 2004. *Agile and Iterative Development: A Manager's Guide*. Boston, MA: Addison-Wesley.

Lucidchart (2024) *UML Class Diagram Tutorial*, Available at: https://www.lucidchart.com/pages/uml-class-diagram (Accessed: 22 March 2026).

Pressman, R.S., 2014. *Software Engineering: A Practitioner's Approach*. 8th ed. New York: McGraw-Hill.

Satzinger, J.W., Jackson, R.B. and Burd, S.D., 2016. *Systems Analysis and Design in a Changing World*. 7th ed. Boston, MA: Cengage Learning.

Sharp, H., Rogers, Y. and Preece, J., 2019. *Interaction Design: Beyond Human-Computer Interaction*. 5th ed. Indianapolis, IN: John Wiley & Sons.

SmartDraw (2024) *Class Diagram Tutorial*, Available at: https://www.smartdraw.com/class-diagram/ (Accessed: 22 March 2026).

Stair, R.M. and Reynolds, G.W., 2020. *Principles of Information Systems*. 14th ed. Boston, MA: Cengage Learning.

Visual Paradigm (2024) *What is Class Diagram?*, Available at: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/ (Accessed: 22 March 2026).

Wiegers, K. and Beatty, J., 2013. *Software Requirements*. 3rd ed. Redmond, WA: Microsoft Press.